5.1 Non-Comparison-Based Sorting

1. Non-Comparison-Based Sorting

Definition:
Non-comparison-based sorting algorithms sort data **without comparing elements with each other**. Instead, they use properties of the data such as **digit values, frequency, or value ranges**.

Examples

1. Counting Sort
2. Radix Sort
3. Bucket Sort

Main advantage

- Can achieve **linear-time sorting** under suitable assumptions.
- Does not use the comparison decision-tree approach.

---

2. Counting Sort

Definition

Counting Sort is a non-comparison-based sorting algorithm that sorts elements by **counting the occurrences of each distinct element** and using these counts to determine their positions in the sorted output. It is efficient for integers within a known range.

Basic Assumption

- Input contains integers in a known range **0 to k**.
- n = number of input elements.
- k = highest value/range parameter.

Arrays Used

1. **Input Array A[1..n]** → stores input elements.
2. **Output Array B[1..n]** → stores sorted elements.
3. **Temporary Array C[1..k]** → stores counts temporarily.

Working

1. Find the range of input values.
2. Initialize the count array.
3. Count the occurrence of every value.
4. Convert counts into cumulative counts.
5. Place each element into its correct position in the output array.
6. Obtain the sorted array.

Time Complexity

$$\boxed{O(n+k)}$$

Where:

- `n` = number of elements
- `k` = range/highest value

Space Complexity

$$\boxed{O(n+k)}$$

When Counting Sort performs poorly

Counting Sort performs poorly when the **range of values is very large compared with the number of elements**.

Example idea:

```
[5, 1000000, 5000000]
```

Here, only a few elements exist, but a very large count array would be required.

Advantages

- Fast for integers with a small known range.
- Can achieve linear time.
- Does not use element comparisons.
- Stable when implemented using cumulative counts.

Disadvantages

- Requires extra memory.
- Inefficient when the value range `k` is very large.
- Mainly suitable for integer/key values within a manageable range.

---

3. Radix Sort

Definition

Radix Sort is a non-comparative, stable sorting algorithm that sorts numbers by processing their **individual digits**, generally from the **Least Significant Digit (LSD)** to the **Most Significant Digit (MSD)**.

Key Points

- It is a **generalization of Bucket Sort**.
- Uses several passes of bucket-based sorting.
- Processes one digit at a time.
- Uses a **stable sorting method** at each pass.
- For decimal numbers, buckets are generally `0–9`.

Working

1. Find the maximum number to determine the number of digits.
2. Start from the **LSD (rightmost digit)**.
3. Distribute elements according to the current digit.
4. Collect elements from buckets in order.
5. Move to the next digit.
6. Repeat until the **MSD** is processed.
7. The final list is sorted.

Time Complexity

If:

- `n` = number of elements
- `k` = number of digits/characters
- `r` = radix/number of possible digits

One pass:

$$O(n + r)$$

For `k` passes:

$$\boxed{O(k(n + r))}$$

Space Complexity

Approximately:

$$\boxed{O(n + r)}$$

Advantages

- Linear-time under suitable conditions.
- Stable.
- Does not require comparisons.
- Efficient for integers with fixed number of digits.
- Can also sort strings.

Disadvantages

- Requires extra space.
- Performance depends on number of digits and radix.
- Not ideal for arbitrary data without suitable representation.

Radix Sort for Names/String of Different Lengths

Radix Sort can also be applied to **strings/names** by treating characters as digits.

Method

1. Choose a fixed character order, e.g. alphabetic order.
2. Process characters from **right to left** for LSD Radix Sort.
3. Use a stable sorting method for each character position.
4. For shorter strings, treat the **missing character as a special value smaller than normal characters**.
5. Continue until the maximum string length is processed.

Important: The sorting method used for every character position must be **stable**. The PDF also assumes a stable sub-sort such as Counting Sort.

---

4. Bucket Sort

Definition

Bucket Sort is a non-comparison-based sorting algorithm that distributes input elements into several **buckets/groups according to their values**. Each bucket is then sorted individually, and all sorted buckets are concatenated.

Working

1. **Divide the range** into a fixed number of buckets.
2. **Distribute elements** into appropriate buckets.
3. **Sort each bucket** using Insertion Sort or another sorting algorithm.
4. **Concatenate** all sorted buckets.

Time Complexity

Let:

- `n` = number of elements
- `k` = number of buckets

Case	Complexity
Best	O(n + k)
Average	O(n + k)
Worst	$O(n^2)$

The worst case occurs when **all elements fall into one bucket** and that bucket requires quadratic sorting.

Space Complexity

$$\boxed{O(n+k)}$$

Suitable For

- Data that is **uniformly distributed**.
- Floating-point numbers or integers within a specific range.

Advantages

- Simple and efficient for uniformly distributed data.
- Can achieve approximately linear time.
- Useful for floating-point values in a known range.

Disadvantages

- Performance depends heavily on data distribution.
- Poor distribution can produce a large bucket.
- Requires extra space for buckets.
- Worst case can be `O(n²)`.

5.2 Parallel Sorting

5. Parallel Sorting

Definition

Parallel Sorting divides a sorting problem into smaller subproblems that are processed **simultaneously by multiple processors, cores, or threads**.

Why Required in Multicore Systems?

- Utilizes multiple CPU cores.
- Reduces sorting time.
- Allows large datasets to be processed concurrently.
- Improves scalability and processor utilization.

Two Types of Parallelism

1. Task Parallelism
Different processors/threads work on different parts of the array simultaneously.

2. Data Parallelism
The same operation is applied to different portions of data in parallel.

Challenges

- Synchronization
- Load balancing
- Communication overhead
- Uneven workload distribution

---

6. Parallel Quick Sort

Definition

Parallel Quick Sort is a parallel version of Quick Sort where the **left and right subarrays created after partitioning are processed concurrently** using multiple threads/processes.

Working

1. **Select Pivot**
   Choose a suitable pivot, such as random or median-of-three.
2. **Partition**
   Elements smaller than the pivot go to the left and larger elements go to the right.
3. **Parallel Recursive Sorting**
   Sort the left and right subarrays using separate threads/processes.

4. **Synchronization**
   Wait until the parallel tasks finish.

Complexity

Case	Time
Best/ideal parallel	O(n)
Average with `p` processors	O(n log n / p)
Worst	$O(n^2)$

Problem with Too Many Threads

Creating a new thread for every recursive subarray is inefficient because:

- Thread creation has overhead.
- Too many threads cause **context switching**.
- Processors become overloaded.
- Synchronization overhead increases.
- Performance can become worse instead of better.

Solution: Limit Parallelism

Use only a **limited number of threads/processors**, based on available processing resources.

A practical approach is:

Parallel for large subarrays → Sequential Quick Sort for small subarrays.

Another approach from your PDF is the **work-pool model**, where sublists are maintained as tasks and assigned to available processors instead of continuously creating new processors.

Optimized Parallel Quick Sort

- Use a good pivot such as **random or median-of-three**.
- Partition the array.
- Parallelize sufficiently large subarrays.
- Limit the number of active threads.
- Use sequential sorting for small subarrays.
- Synchronize only when required.
- Maintain good load balancing.

The PDF emphasizes that good pivot selection and efficient process allocation are important because the partition tree may not be balanced and processor utilization can become inefficient.

---

7. Odd-Even Transposition Sort

Definition

Odd-Even Transposition Sort is a parallel variant of Bubble Sort that works through alternating **even and odd phases**.

Even Phase

Even Phase

Compare/exchange:

$$(P_0, P_1), (P_2, P_3), (P_4, P_5), \ldots$$

Odd Phase

Compare/exchange:

$$(P_1, P_2), (P_3, P_4), (P_5, P_6), \ldots$$

Working

1. Perform the **even phase**.
2. Perform the **odd phase**.
3. Repeat alternating phases.
4. Continue until the elements are sorted.

Time Complexity

Parallel implementation:

O(n)\boxed{O(n)}

The PDF notes that although it has no particular advantage over normal Bubble Sort sequentially, its parallel implementation has `O(n)` time.

Key Point to Memorize

Even → Odd → Even → Odd

---

8. Bitonic Merge Sort

Definition

Bitonic Merge Sort is a parallel sorting algorithm based on a **bitonic sequence** and repeated **compare-and-exchange operations**.

Bitonic Sequence

A sequence that first **increases and then decreases**, such as:

$$a_1 < a_2 < \ldots < a_i > a_{i+1} > \ldots > a_n$$

A cyclic shift can also produce a bitonic sequence.

Working

1. Convert the input into a **bitonic sequence**.
2. Compare elements at a fixed distance.
3. Exchange elements if they are in the wrong order.
4. Reduce the comparison distance.
5. Repeat recursively.
6. Obtain the final sorted sequence.

The important property is that compare-and-exchange operations divide a bitonic sequence into two bitonic sequences where all elements of one part are smaller than those of the other.

Phases

For `n = 2^k`:

- Phase 1: Create small bitonic sequences.
- Intermediate phases: Merge them into larger bitonic sequences.
- Final phase: Sort the complete bitonic sequence.

Time Complexity

$$\boxed{O(\log^2 n)}$$

Advantages

- Highly parallel.
- Predictable execution structure.
- Suitable for parallel processors.
- Uses compare-and-exchange operations.

Disadvantage

- Mainly suitable when the number of elements is a power of two.
- More compare operations than some sequential sorting algorithms.

---

9. Parallel Merge Sort

Definition

Parallel Merge Sort divides the array into smaller subarrays, sorts them **concurrently**, and then merges the sorted subarrays. It can use multiple processors or threads.

Steps

Divide → Conquer → Merge

1. Divide the input among processors.
2. Sort each subarray independently.
3. Merge sorted subarrays.
4. Merging can also be performed in parallel.



where p is the number of processors.

Advantages

- Faster execution.
- Good scalability.
- Suitable for large datasets and multicore systems.

Challenges

- Synchronization.

- Communication overhead.
- Load balancing.

---

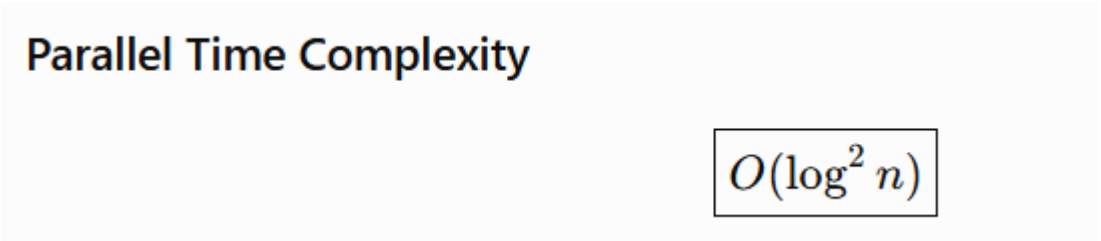
10. Odd-Even Merge Sort

Definition

Odd-Even Merge Sort is a parallel sorting algorithm that recursively applies the **odd-even merge algorithm** to merge sorted sublists.

Working

1. Sort small sublists.
2. Merge odd-position elements.
3. Merge even-position elements.
4. Perform compare-and-exchange on neighboring elements.
5. Repeat recursively until the complete list is sorted.



5.3 External Sorting

⚠️ **Important:** Your uploaded PDF only lists **"External sorting and its applications in large-scale data engineering"** as a chapter topic and student presentation; it does not provide the actual external-sorting lecture notes.

So the following is a **standard exam-oriented note**, not directly extracted from the PDF.

External Sorting

Definition

External Sorting is a sorting technique used when the dataset is **too large to fit completely into main memory (RAM)**. It uses **secondary storage such as disks/SSDs** to store and process the data.

Need for External Sorting

- Dataset is larger than available RAM.
- Large files/databases need to be sorted.
- Used in large-scale data processing.
- Reduces the need to load the entire dataset into memory.

Basic Working

External sorting generally uses two main phases:

1. Run Formation

- Read manageable portions of data into RAM.
- Sort each portion internally.
- Store each sorted portion as a **sorted run** on disk.

2. Merge Phase

- Read multiple sorted runs.
- Merge them into larger sorted runs.
- Continue until one completely sorted file is produced.

Common Algorithm

External Merge Sort

Advantages

- Can sort datasets much larger than RAM.
- Suitable for large files and databases.
- Efficient for sequential disk access.
- Widely used in large-scale data processing.

Disadvantages

- Disk I/O is slower than RAM access.
- Requires additional storage.
- Multiple read/write operations increase processing time.
- Performance depends heavily on I/O efficiency.

Applications

- Database systems
- Large file sorting
- Data warehouses
- Big-data processing
- Large-scale data engineering

- External database operations

⭐ Most Important Definitions to Memorize

Topic	One-line definition
Counting Sort	Sorts elements by counting their occurrences and using counts to determine their positions.
Radix Sort	Sorts elements by processing individual digits/characters from LSD to MSD.
Bucket Sort	Distributes elements into buckets, sorts each bucket, and concatenates them.
Parallel Sorting	Sorts data by processing different parts simultaneously using multiple processors/threads.
Parallel Quick Sort	Sorts Quick Sort's subarrays concurrently using multiple threads/processes.
Odd-Even Transposition Sort	Parallel Bubble Sort that alternates between even and odd compare-exchange phases.
Bitonic Merge Sort	Parallel sorting algorithm that creates a bitonic sequence and recursively applies compare-exchange operations.
Parallel Merge Sort	Divides data into subarrays, sorts them concurrently, and merges the results.
External Sorting	Sorts datasets too large to fit in RAM by using secondary storage.

⭐ Complexity Table

Algorithm	Main Time Complexity
Counting Sort	$O(n + k)$
Bucket Sort	Average $O(n+k)$, Worst $O(n^2)$
Radix Sort	$O(k(n+r))$
Parallel Merge Sort	$O(n \log n / p)$
Parallel Quick Sort	Average $O(n \log n / p)$
Odd-Even Transposition	$O(n)$ parallel
Odd-Even Merge Sort	$O(\log^2 n)$
Bitonic Sort	$O(\log^2 n)$